SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I DARANI E (192312120) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

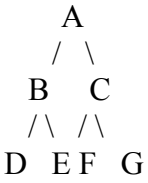
Signature of the student:
DARANI E

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

Answer:

Breadth-First Search (BFS)

Given graph: A is the root; A has children B and C; B has children D and E; C has children F and G. BFS visits nodes level by level. The left child is visited before the right child.

Queue is FIFO (First In, First Out). In each iteration, remove the front node, mark it visited, and add its unvisited children to the rear of the queue.

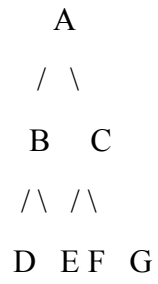
Iteration	Queue before expansion	Visited node	Queue after expansion
1	[A]	A	[B, C]
2	[B, C]	B	[C, D, E]
3	[C, D, E]	C	[D, E, F, G]
4	[D, E, F, G]	D	[E, F, G]

5	[E, F, G]	E	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[]

1. BFS traversal order: A → B → C → D → E → F → G.

2. Queue contents after each iteration: [B,C] → [C,D,E] → [D,E,F,G] → [E,F,G] → [F,G] → [G] → [].

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Answer

Depth-First Search (DFS)

DFS explores one branch as deeply as possible before backtracking. Starting at A and choosing the left child before the right child, the traversal is A → B → D → E → C → F → G.

Stack dry run (top of stack is shown at the right). To preserve left-to-right visiting order, push the right child first and then the left child.

Iteration	Stack before pop	Visited node	Stack after expansion
1	[A]	A	[C, B]
2	[C, B]	B	[C, E, D]
3	[C, E, D]	D	[C, E]
4	[C, E]	E	[C]
5	[C]	C	[G, F]
6	[G, F]	F	[G]

7	[G]	G	[]

DFS traversal order: A → B → D → E → C → F → G.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.
Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

Answer:

8-Puzzle Problem Using BFS

Initial state:

```
1 3 6

5 0 2

4 7 8
```

Goal state:

```
1 2 3

4 5 6
```

7 8 0

Convention used for the dry run: the blank (0) is moved in the order Up, Down, Left, Right. Repeated states are not added to the queue. Each state is written as three rows separated by "/". For example, 136/502/478 represents the initial board.

BFS expands the oldest state in the FIFO queue. The queue table below shows the complete queue in compact form before and after each of the first five expansions.

Iteration	Current state expanded	Queue before expansion	Queue after expansion
1	136/502/478	[136/502/478]	[106/532/478, 136/572/408, 136/052/478, 136/520/478]
2	106/532/478	[106/532/478, 136/572/408, 136/052/478, 136/520/478]	[136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478]
3	136/572/408	[136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478]	[136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480]
4	136/052/478	[136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480]	[136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078]
5	136/520/478	[136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078]	[016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078, 130/526/478, 136/528/470]

Note: In the first five iterations, the current state is removed from the front of the queue. Its legal, previously unseen successor states are appended at the rear.

Answers to the 8-puzzle questions

1. Nodes/states expanded in the first five iterations:

Iteration 1: 136/502/478

Iteration 2: 106/532/478

Iteration 3: 136/572/408

Iteration 4: 136/052/478

Iteration 5: 136/520/478

2. Total states generated: 500 distinct states are discovered (including the initial state and the goal) before the goal is selected for expansion, using the stated Up–Down–Left–Right successor order and duplicate-state checking. The goal is reached as the 311th expanded state. Counts can vary if a different successor order or a different definition of "generated" is used.

3. Complete shortest solution path (each board is a state; 0 is the blank):

Step	State / move
Initial	136/502/478
Move blank Right	136/520/478
Move blank Up	130/526/478
Move blank Left	103/526/478
Move blank Down	123/506/478
Move blank Left	123/056/478
Move blank Down	123/456/078
Move blank Right	123/456/708
Move blank Right – Goal	123/456/780

Move sequence: Right → Up → Left → Down → Left → Down → Right → Right. Total path cost = 8 moves, since each blank movement has unit cost.

4. BFS finds the shortest solution in an unweighted 8-puzzle because every legal move has the same cost (1). BFS explores all states at depth 0, then depth 1, then depth 2, and so on. Therefore, the first time it reaches the goal, that goal has the minimum number of moves among all possible solutions.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.